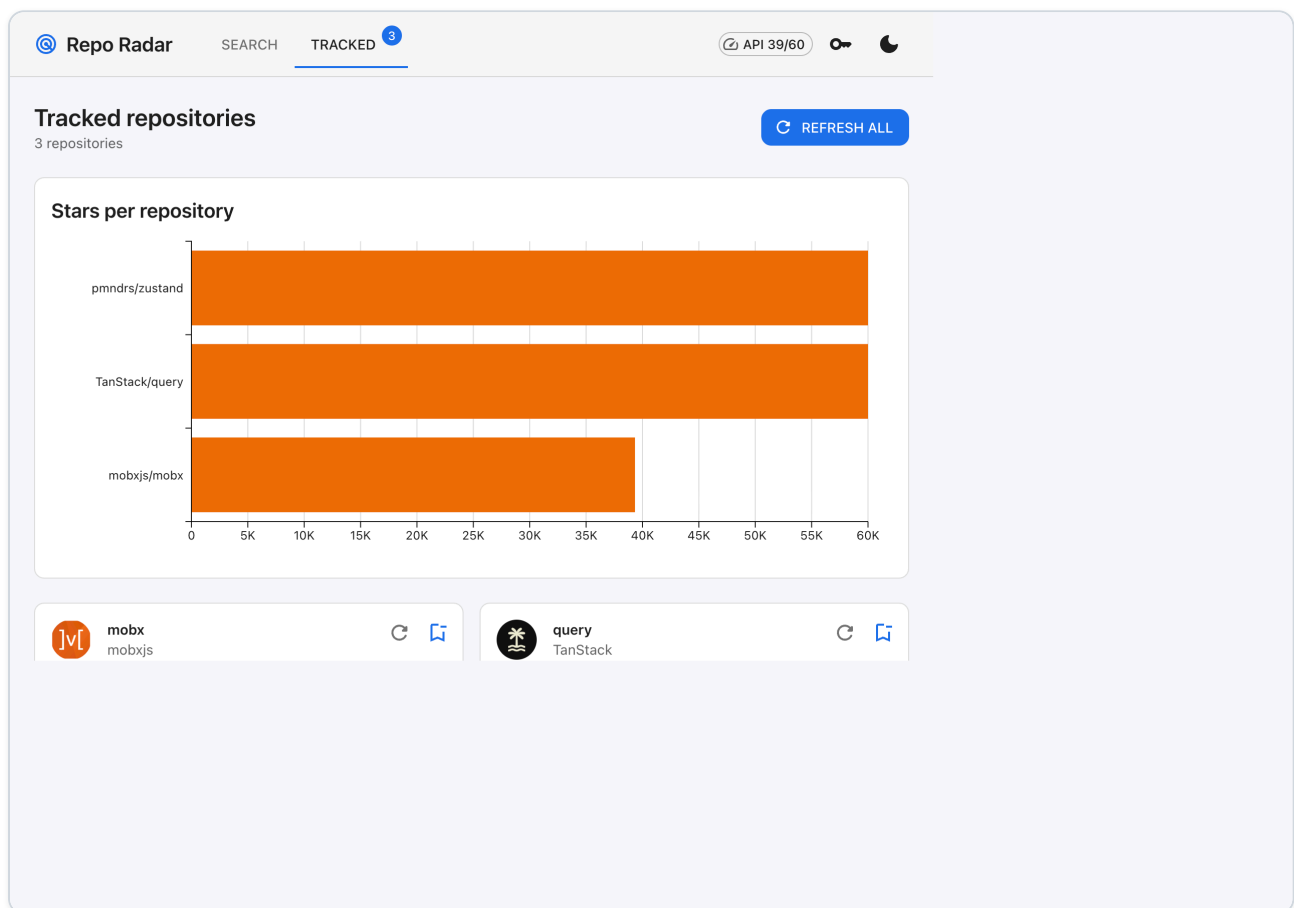
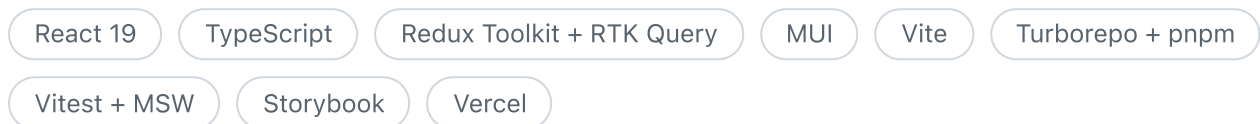


Repo Radar

Project Walkthrough — Part 1: Structure & Architecture

A dashboard to search GitHub repositories, track favourites, and monitor stars, open issues and last-commit dates. Built as a Siemens senior front-end technical task.



The Tracked view: stars chart, uniform repo cards, per-repo refresh and untrack.

Contents

1. The big picture: a layered monorepo
2. Why split into packages at all?
3. The dependency graph — a strict DAG
4. Package by package
 - `core` — domain layer
 - `store` — data layer
 - `ui` — presentation layer
 - `web` — composition layer
 - Shared tooling packages
5. Why Turborepo + pnpm
6. How a request flows through the layers
7. Cheat-sheet of named patterns

1. The big picture: a layered monorepo

```
repo-radar/
├─ apps/
│   └─ web/                ← the only deployable thing
├─ packages/
│   ├─ core/               ← pure TypeScript, zero framework deps
│   ├─ store/              ← data layer (Redux Toolkit + RTK Query)
│   ├─ ui/                 ← presentational MUI components
│   ├─ eslint-config/      ← shared tooling config
│   └─ typescript-config/  ← shared tooling config
├─ turbo.json
├─ pnpm-workspace.yaml
└─ vercel.json
```

Pattern: Monorepo with an `apps/` + `packages/` split (the Turborepo convention). The rule is simple: `apps/` contains things you *deploy*; `packages/` contains things you *import*.

There is exactly one app today, but the shape lets you add `apps/storybook-site`, `apps/mobile`, or `apps/api` later without reshuffling anything.

2. Why split into packages at all?

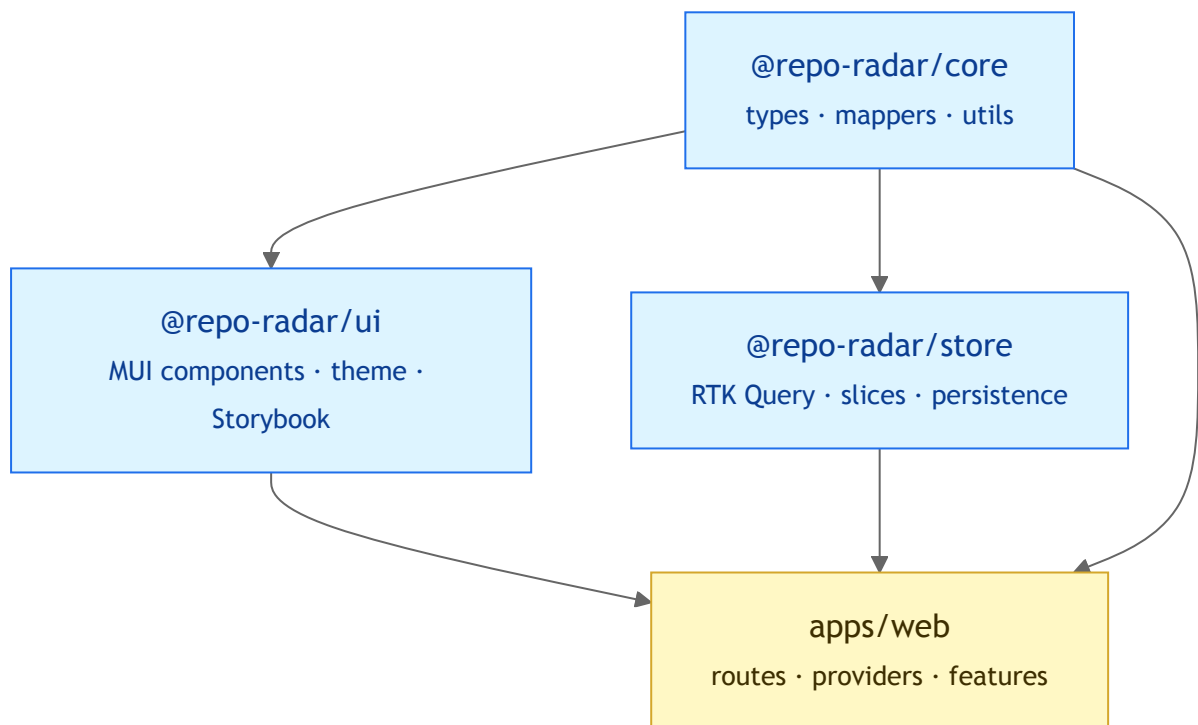
The brief said reviewers would judge “*monorepo structure and separation of concerns*” and “*reusable/shared components and packages*.” A single `src/` folder with subfolders technically

works, but **package boundaries are enforced by the tooling, not by convention**:

- If `ui` tries to `import { useAppSelector } from '@repo-radar/store'`, it fails — `store` isn't in `ui`'s `package.json` dependencies. A folder split can't stop that; a package split does.
- Each package has its own `tsconfig`, `eslint.config` and tests, and can be type-checked / tested / cached **independently** by Turborepo.

Pattern: Enforced module boundaries (a.k.a. "package-based architecture" or "modular monolith"). The point is making the wrong import *impossible* rather than merely discouraged.

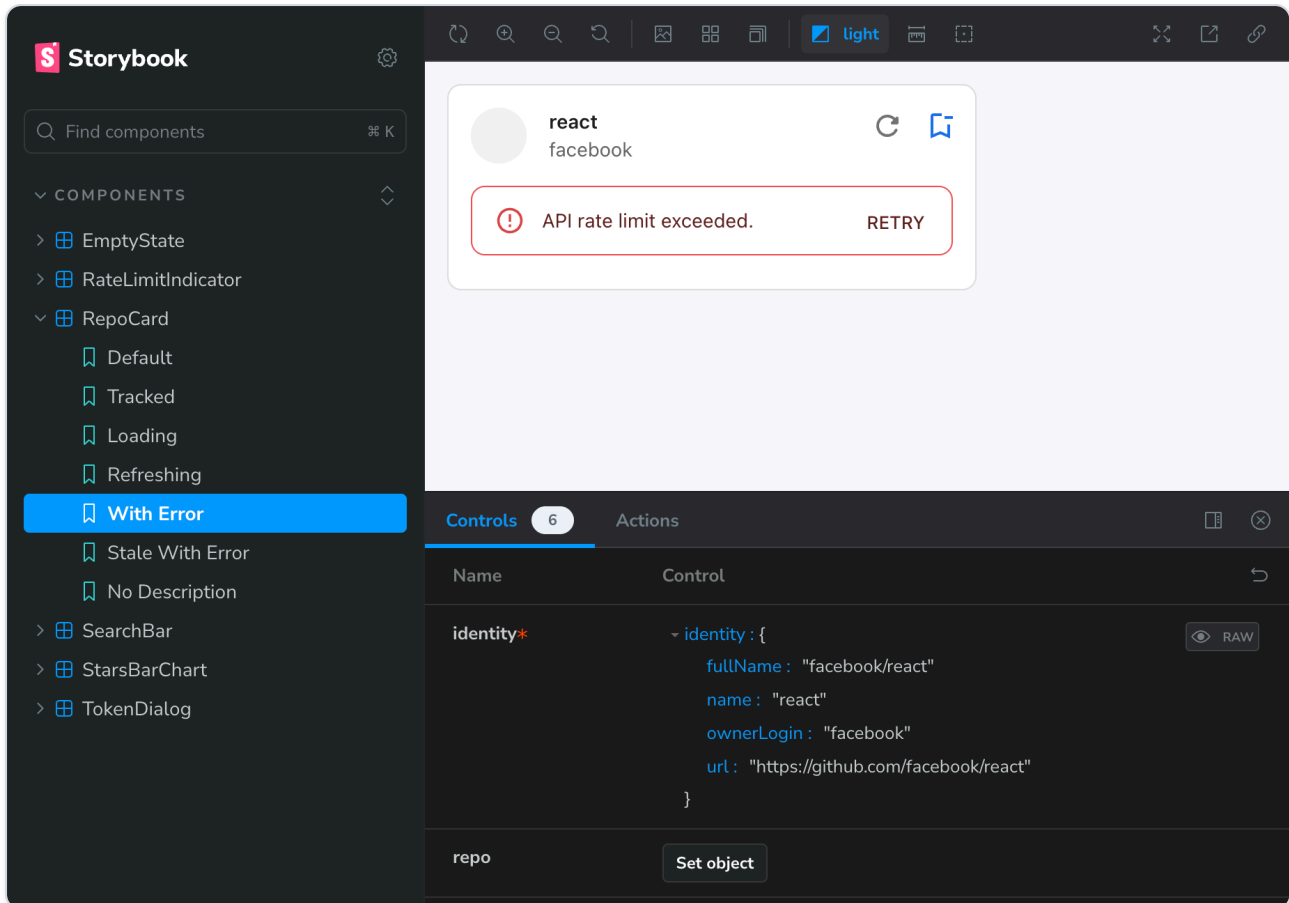
3. The dependency graph — a strict DAG



Dependencies flow in **one direction only**: `core` knows nothing, `store` and `ui` know only `core`, and `web` knows everything. `store` and `ui` are **siblings** that never import each other.

Pattern: Directed Acyclic Graph (DAG) of dependencies / the Dependency Rule — from Clean Architecture: "source code dependencies must point only inward toward higher-level policies." `core` is the innermost ring.

Why does `ui` not depend on `store`? Because that is what makes `ui` **reusable**. `RepoCard` doesn't know Redux exists — it takes `repo`, `isLoading`, `onRefresh` as props. You can render it in Storybook with fake data, test it without a store, or drop it into a Next.js app using a totally different state library.



Proof of decoupling: `RepoCard` renders every state (Default, Tracked, Loading, Refreshing, With Error, Stale With Error, No Description) in Storybook with no Redux store and no network.

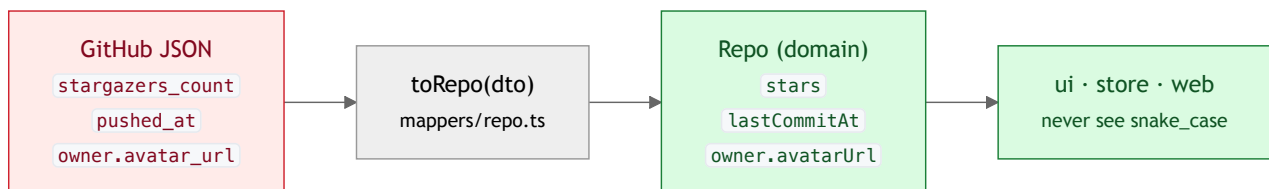
4. Package by package

4.1 `packages/core` — the domain layer

```
core/src/
├─ constants.ts      # API base URL, debounce ms, storage keys, schema version
├─ types/
│   ├─ github.ts     # GitHubRepoDto, GitHubSearchResponseDto (raw API, snake_case)
│   └─ domain.ts     # Repo, TrackedRepo, RateLimitInfo, ThemeMode (ours, camelCase)
├─ mappers/repo.ts   # toRepo(dto), toRepoSearchResult(dto), toTrackedRepo(repo)
└─ utils/
    ├─ format.ts     # formatCompactNumber, formatRelativeDate, formatCountdown
    ├─ storage.ts    # loadFromStorage / saveToStorage with versioned envelope
    └─ guards.ts     # isTrackedRepo, isTrackedRepoList, isThemeMode (runtime checks)
```

Zero dependencies on React, Redux or MUI. It runs in Node, in the browser, in a test — anywhere. Three named patterns live here.

a) DTO vs Domain Model, joined by a Mapper



`GitHubRepoDto` is *their* shape. `Repo` is *our* shape, containing only fields we actually use. The mapper translates once, at the boundary.

- If GitHub renames a field, one file changes.
- If you swap to GraphQL or a backend proxy, the entire UI is untouched.
- UI, tests and stories all work with the clean domain type.

Pattern: Anti-Corruption Layer (Domain-Driven Design). You don't let an external system's model leak into yours.

b) Runtime type guards at system boundaries

TypeScript types vanish at runtime. Data from `localStorage` is `unknown` — it could be corrupt, from an old version, or hand-edited. `isTrackedRepoList(value): value is TrackedRepo[]` checks the shape at runtime and TypeScript narrows the type when it returns `true`.

Rule of thumb: **validate at system boundaries** (network, storage, URL params); trust types internally.

c) Versioned storage envelope

```
{ "version": 1, "data": [ { "id": 32215970, "fullName": "mobxjs/mobx", ... } ] }
```

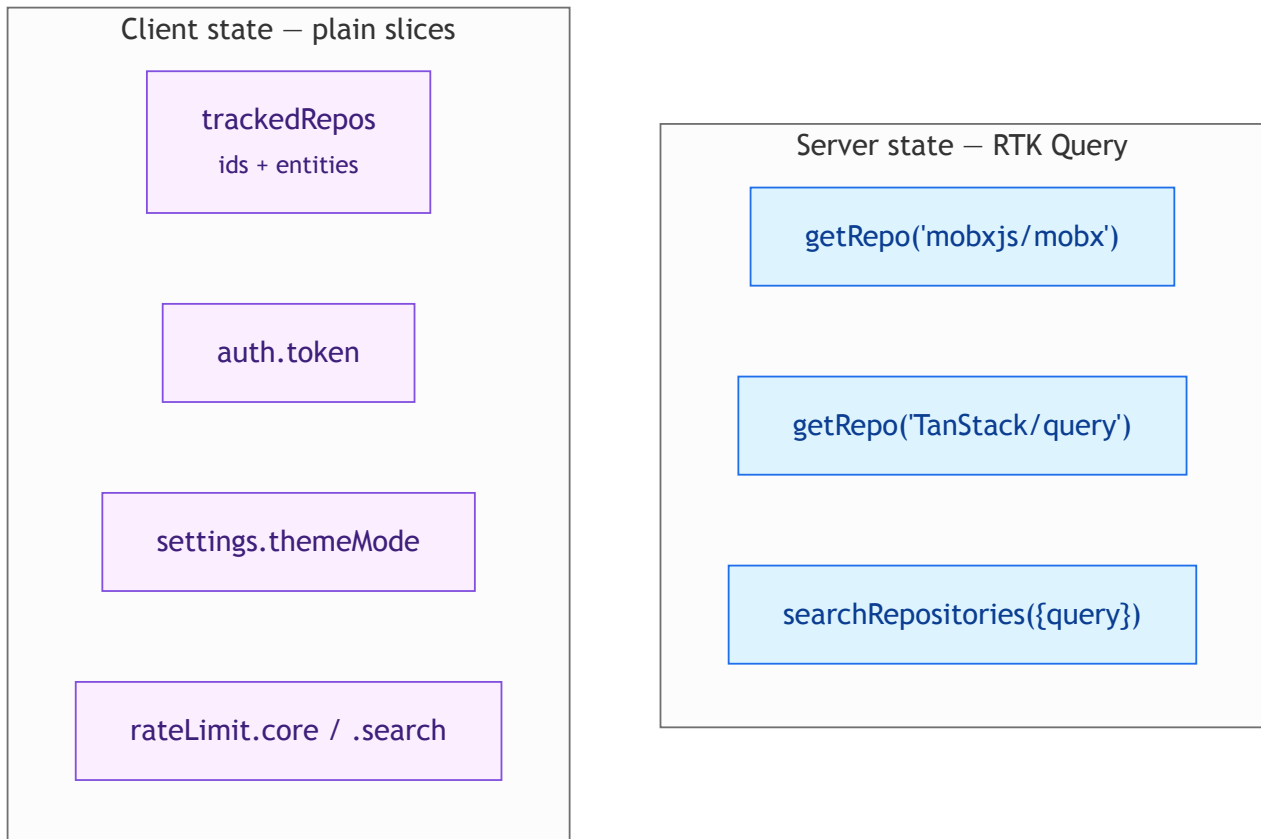
On load, if `version` doesn't match or the guard rejects `data`, the entry is wiped and the app starts clean instead of crashing.

Pattern: Schema versioning / lightweight migration strategy. Bump the version whenever the persisted shape changes.

4.2 `packages/store` — the data layer

```
store/src/
├─ api/
│   ├─ baseQuery.ts           # wraps fetchBaseQuery: token, rate-limit headers, error
│   ├─ githubApi.ts          # createApi: searchRepositories, getRepo
│   ├─ apiError.ts           # ApiError + toApiError(): 403/429→rate-limit, 404→not-found
│   └─ rateLimitHeaders.ts    # parse x-ratelimit-* headers
├─ slices/
│   ├─ trackedReposSlice.ts   # createEntityAdapter – what the user tracks
│   ├─ authSlice.ts           # optional PAT
│   ├─ settingsSlice.ts       # theme mode
│   └─ rateLimitSlice.ts      # per-resource budget (core vs search)
├─ persistence/
│   ├─ persistenceMiddleware.ts # listener middleware → storage on specific actions
│   └─ storage.ts             # loadPersistedState / persist* helpers
├─ thunks/refreshAllTrackedRepos.ts
├─ rootReducer.ts
├─ store.ts                   # createStore({ storage, persist, preloadedState })
├─ selectors.ts
├─ hooks.ts                   # useAppDispatch / useAppSelector (typed)
└─ index.ts                   # public API – the ONLY thing other packages import
```

a) Server state vs client state



This is the most important conceptual split in modern React apps. *Server state* (repo data) is cached, has loading/error states, goes stale, needs refetching. *Client state* (what I track, my theme, my token) is owned by the user and has none of those concerns. RTK Query handles the first; plain slices handle the second. Copying fetched data into a slice is a classic anti-pattern that produces stale-data bugs.

b) Normalised state with `createEntityAdapter`

```
trackedRepos: {
  ids: [32215970, 207645083, 234898203],
  entities: {
    32215970: { id: 32215970, fullName: "mobxjs/mobx", ... },
    207645083: { id: 207645083, fullName: "TanStack/query", ... },
    ...
  }
}
```

A normalised shape — like a database table with a primary key. `selectIsTracked(id)` is an $O(1)$ object lookup rather than an $O(n)$ `array.find`. The adapter also provides `addOne` / `removeOne` / `setAll` reducers and memoised selectors for free.

c) Store factory instead of a singleton

`createAppStore()` is a *function*, not a pre-built `export const store`. Tests create a fresh, isolated store per test with an in-memory storage adapter; the app calls it once in `apps/web/src/app/store.ts`.

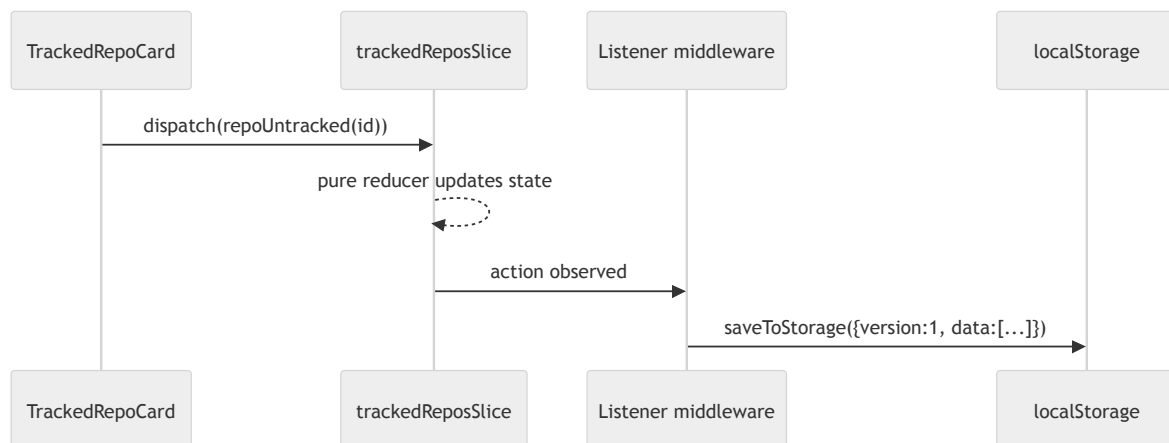
Pattern: Factory function + Dependency Injection. The `StorageAdapter` is injected, so persistence is testable without touching real `localStorage`.

d) Facade via the `index.ts` barrel

Only what's exported from `store/src/index.ts` is public. `rootReducer.ts`, `baseQuery.ts` etc. are implementation details.

Pattern: Facade at package level — consumers get a small, stable surface area.

e) Side effects via `createListenerMiddleware`



Persistence doesn't live inside reducers (reducers must be pure). A listener says "when `repoTracked`, `repoUntracked` or `allReposUntracked` fires, write the list to storage." "What changed" stays separate from "what to do about it."

Pattern: Listener / effect middleware — a lightweight saga-style side-effect layer.

4.3 packages/ui — the presentation layer

```
ui/src/
├── theme/
│   ├── createAppTheme.ts      # MUI theme factory (light | dark)
│   └── AppThemeProvider.tsx
├── components/
│   ├── RepoCard/              # RepoCard.tsx + .test.tsx + .stories.tsx
│   ├── SearchBar/
│   ├── StarsBarChart/
│   ├── RateLimitIndicator/
│   ├── TokenDialog/
│   └── EmptyState/ ErrorBanner/ RepoGrid/ StatChip/ RepoStatsRow/ ThemeToggle/ AppShe
└── index.ts
```

**query**
TanStack

Powerful asynchronous state management, server-state utilities and data fetching for the web. TS/JS, React Query,...

★ 50.3K ⓘ 169 ⌚ 4 hours ago

TypeScript

`RepoCard` — stars, open issues, last commit, language, refresh + track actions.

GitHub access token

Unauthenticated requests are limited to 60 per hour. Add a [fine-grained personal access token](#) with no extra scopes to raise the limit to 5,000 per hour.

Personal access token

github_pat_...

ⓘ

The token is stored only in this browser's localStorage and sent directly to api.github.com. It is never sent anywhere else.

CANCEL

SAVE

`TokenDialog` — opt-in personal access token, stored only in the user's browser.

a) Presentational vs Container components

Every component in `ui` is *presentational*: it receives data and callbacks via props and has no idea where they come from. The *containers* live in `apps/web/src/features/` — `TrackedRepoCard` calls `useGetRepoQuery` and passes the result into `RepoCard`.

Pattern: Presentational and Container Components (Dan Abramov) — in modern terms, "dumb components + smart hooks."

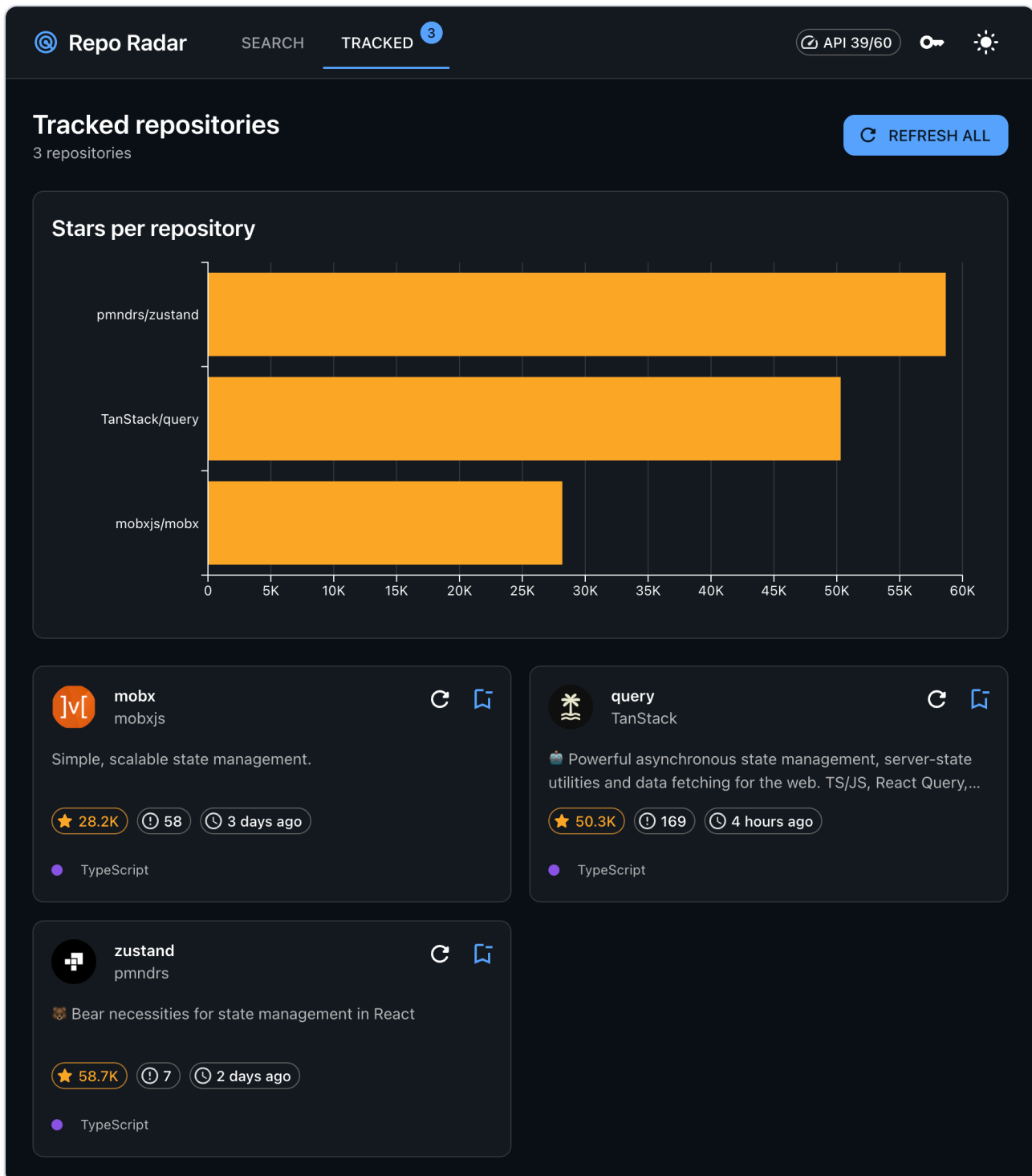
b) Co-location

`RepoCard.tsx`, `RepoCard.test.tsx` and `RepoCard.stories.tsx` sit in one folder. Change the component and its test and story are right there; delete the folder and everything related goes with it.

Pattern: Component-folder co-location.

c) Theming and a uniform grid

`createAppTheme(mode)` is the single source of colours, radii and typography. Cards are forced to identical dimensions (`width: 100%`, `height: 100%`, fixed two-line description, always-present footer) so the grid reads as a consistent system regardless of content length.



The same components under the dark theme — one theme factory, zero component changes.

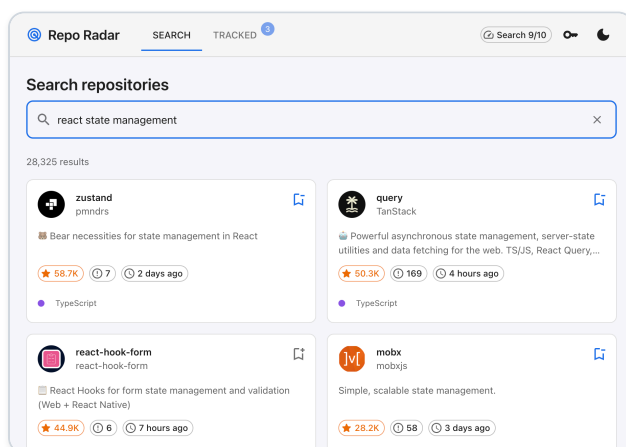
4.4 apps/web — the composition layer

```
web/src/
├─ main.tsx           # ReactDOM root
├─ App.tsx            # AppShell + lazy Routes
├─ app/
│   ├─ AppProviders.tsx # Redux Provider → Theme → ErrorBoundary
│   ├─ store.ts         # the single app store instance
│   └─ routes.ts
├─ components/ErrorBoundary.tsx · NavTabs.tsx
├─ features/          # ← the key folder
│   ├─ search/        # SearchPage, SearchResultCard
│   ├─ tracked/        # TrackedPage, TrackedRepoCard, RefreshAllButton, Tracked
│   ├─ settings/      # TokenSettings
│   ├─ rate-limit/    # RateLimitBadge
│   └─ theme/         # ThemeModeToggle
├─ hooks/             # useDebounceValue, useTrackToggle
└─ lib/toUserMessage.ts # ApiError → human-readable copy
```

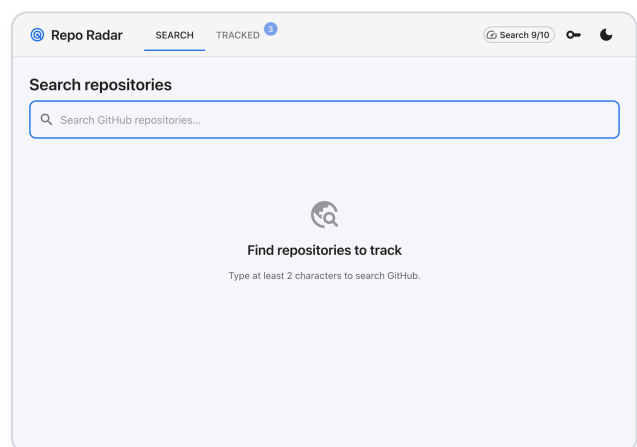
a) Feature-based folders

Instead of grouping by type (`components/` , `hooks/` , `containers/`), code is grouped by *feature* (`search/` , `tracked/`). Everything about the Tracked page is in one folder. Adding a feature adds a folder; removing one removes a folder.

Pattern: Feature-based structure / vertical slicing — the core idea behind Feature-Sliced Design and the “bulletproof-react” layout.



`features/search` — debounced search, load-more pagination.



Explicit empty state before a query reaches the minimum length.

b) Thin containers

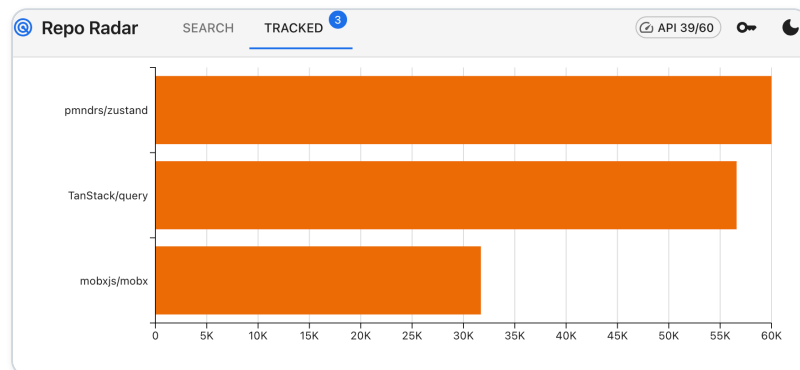
Feature components are deliberately small — usually one hook call plus one `ui` component.

`TrackedRepoCard` is ~20 lines: call `useGetRepoQuery`, map the result to `RepoCard` props. Logic lives in `store`; rendering lives in `ui`; `web` just plugs them together.

If `web` starts growing large components, that's a smell that logic belongs in a lower layer.

c) Route-based code splitting

`SearchPage` and `TrackedPage` are `React.lazy()` imports. The ~300 kB charts library only downloads when someone visits `/tracked`; Vite emits it as a separate chunk.



`StarsBarChart` reads straight from the RTK Query cache — no extra requests.

d) Composition Root

`AppProviders.tsx` is where every provider (Redux, MUI theme, error boundary) is assembled — the one place that knows about all the layers. Everything below it is unaware of the wiring.

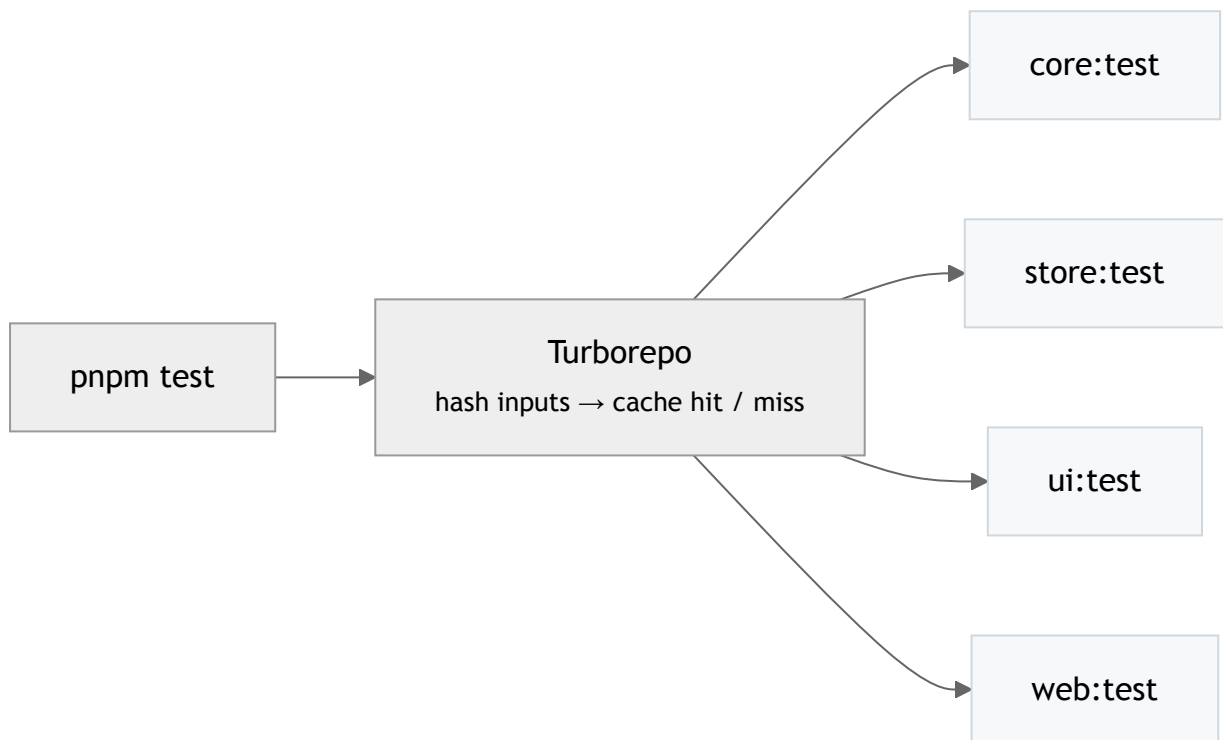
Pattern: Composition Root (Dependency-Injection terminology).

4.5 `eslint-config` & `typescript-config`

Shared tooling as packages. Every other package does `"extends": "@repo-radar/typescript-config/react-library.json"`. Change a compiler flag or lint rule once and it applies everywhere — the standard Turborepo pattern.

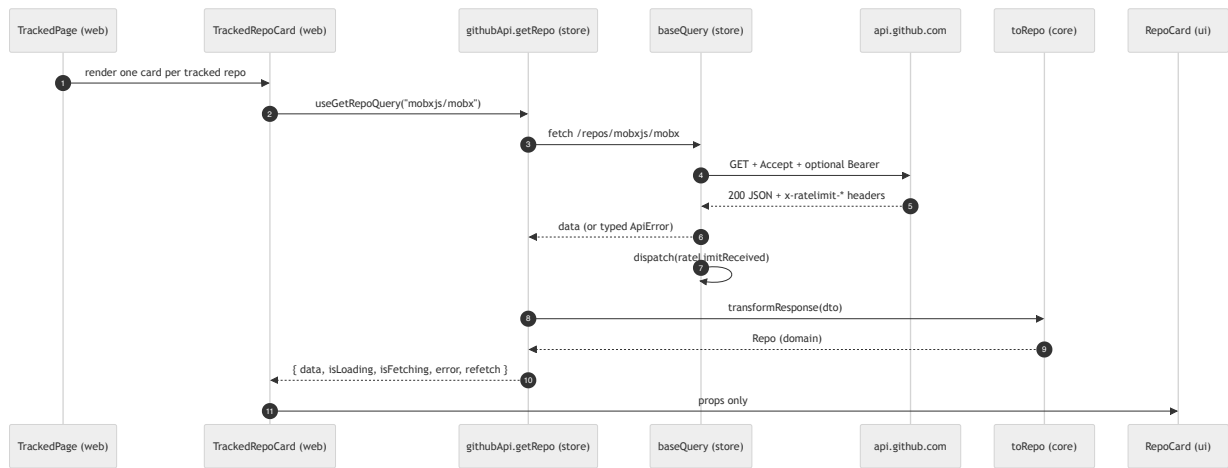
5. Why Turborepo + pnpm

- **pnpm workspaces** — the `workspace:*` protocol links packages by symlink. One `node_modules` at the root, no duplicate React copies (which would break hooks).
- **Turborepo** — builds the task graph from `package.json` dependencies, runs tasks in parallel where possible, and **caches by content hash**. Run `pnpm test` twice without changes and the second run is instant. `dependsOn: ["^build"]` means "build my dependencies first."

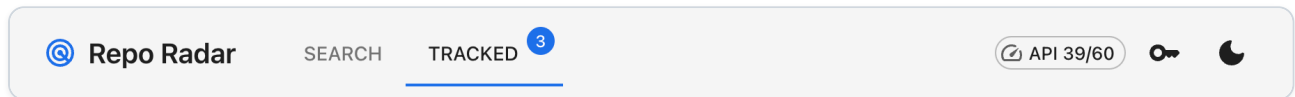


6. How a request flows through the layers

Tracing one interaction — the user opens the Tracked page — shows every layer doing exactly one job:



Each tracked card owns its own cache entry, so `isLoading`, `error` and `refetch()` are isolated per repo with zero custom bookkeeping — this is how the “independent loading and error states per repo” requirement is satisfied for free.



Side-effect of the same flow: the `x-ratelimit-*` headers captured in `baseQuery` drive the live **API 39/60** budget badge.

7. Cheat-sheet of named patterns

Pattern	Where it lives
Monorepo, <code>apps/</code> + <code>packages/</code> split	repo root
Enforced module boundaries / modular monolith	<code>package.json</code> dependencies
Dependency Rule / acyclic dependency graph	<code>core</code> $\leftarrow$ <code>store</code> , <code>ui</code> $\leftarrow$ <code>web</code>
DTO $\rightarrow$ Domain Model via Mapper (Anti-Corruption Layer)	<code>core/mappers</code>
Runtime type guards at boundaries	<code>core/utils/guards.ts</code>
Versioned storage envelope (schema migration)	<code>core/utils/storage.ts</code>
Server state vs client state	RTK Query vs slices
Normalised state (<code>createEntityAdapter</code>)	<code>trackedReposSlice</code>
Factory + Dependency Injection	<code>createAppStore({ storage })</code>
Facade / barrel export	every <code>index.ts</code>
Listener middleware for side effects	<code>persistenceMiddleware</code>
Presentational vs Container components	<code>ui</code> vs <code>web/features</code>
Co-location (component folder)	<code>ui/components/*</code>
Feature-based folders (vertical slicing)	<code>web/features/*</code>
Composition Root	<code>AppProviders.tsx</code>
Route-based code splitting	<code>App.tsx</code> lazy routes

Part 1 of the Repo Radar walkthrough. Upcoming parts: (2) the data layer in depth — RTK Query internals, the base query, error normalisation, refresh-all; (3) UI layer and theming; (4) testing strategy; (5) tooling, CI and deployment.